

Software Architecture

Architecture Principles

Bearbeitungshinweise

Ingo Arnold

2025

Inhaltsverzeichnis

1	Object-Orientation	3
2	Architecture Principles	3
3	Single Responsibility Principle	4
4	Open / Closed Principle	5
5	Liskov Substitution Principle.....	6
6	Interface Segregation Principle	6
7	Dependency Inversion Principle	7
8	Overall Principles Questions	7

1 Object-Orientation

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What is polymorphism?
 - A. The ability of a function to behave differently based on the number of arguments
 - B. Defining methods with the same name in different classes
 - C. Objects of different types can be accessed through the same interface
 - D. A method that does not return any value

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- [*Software Architecture: A Comprehensive Framework and Guide for Practitioners*](#) [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.2.2 Object Orientation
- den unter [*Bibliography*](#) aufgeführten Literaturhinweisen im Foliensatz.

2 Architecture Principles

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- Which of the following is an example of loose coupling?
 - A. Tight dependencies between components
 - B. Use of interfaces to interact between modules
 - C. Hard-coded references between classes
 - D. Global variables shared across modules
- What is a key advantage of information hiding in software architecture?
 - A. Increased transparency of implementation details
 - B. Faster runtime performance
 - C. Easier maintenance and modification of modules
 - D. Reduced need for documentation
- Which of these principles enhances the ability to change software without affecting other modules?
 - A. Modularity
 - B. Tight Coupling

- D. Low Cohesion
- E. Encapsulation

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- [Software Architecture: A Comprehensive Framework and Guide for Practitioners](#) [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.1 Architecture Principles
- den unter [Bibliography](#) aufgeführten Literaturhinweisen im Foliensatz.

3 Single Responsibility Principle

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- According to the Single Responsibility Principle, a class should:
 - A. Handle multiple responsibilities to be more efficient
 - B. Have only one reason to change
 - C. Always include input/output operations
 - D. Implement multiple interfaces
- What is the risk of violating the Single Responsibility Principle?
 - A. Easier maintenance
 - B. More cohesive classes
 - C. Increased coupling between classes
 - D. Faster performance
- Which of the following is an example of a class adhering to SRP?
 - A. *Logger* class that logs messages and handles user input
 - B. *UserService* class that manages user authentication and email notifications
 - C. *PaymentProcessor* class that handles payment processing only
 - D. *Customer* class that manages both customer data and order processing

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

4 Open / Closed Principle

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What does the Open/Closed Principle state?
 - A. A class should be open for extension and open for modification
 - B. A class should be closed for extension and open for modification
 - C. A class should be open for extension but closed for modification
 - D. A class should be closed for both extension and modification
- Which of the following is a common way to adhere to the Open/Closed Principle?
 - A. Using conditional statements within a class
 - B. Extending classes through inheritance
 - C. Using global variables
 - D. Directly modifying class attributes
- How can the Open/Closed Principle help in software development?
 - A. It makes the code less flexible
 - B. It allows changes without affecting existing functionality
 - C. It reduces the number of classes
 - D. It forces developers to rewrite entire modules
- The Open/Closed Principle is violated if:
 - A. You add a new class for a new feature
 - B. You modify existing methods to add new behavior
 - C. You extend a class with new methods
 - D. You use interfaces to define behavior

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

5 Liskov Substitution Principle

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- The Liskov Substitution Principle aims to:
 - A. Ensure subclasses can replace super classes without breaking functionality/semantics
 - B. Limit the use of inheritance
 - C. Reduce the number of subclasses
 - D. Force the use of abstract classes
- A method that throws an exception when a superclass method does not is:
 - A. Adhering to LSP
 - B. Violating LSP
 - C. Irrelevant to LSP
 - D. Partially following LSP

6 Interface Segregation Principle

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What is the main goal of the Interface Segregation Principle (ISP)?
 - A. To reduce the number of classes in a project
 - B. To make interfaces large enough to handle many behaviours
 - C. To ensure that clients do not depend on methods they do not use
 - D. To prevent the use of abstract classes
- How can the Interface Segregation Principle improve software design?
 - A. By making interfaces more generic and easier to implement
 - B. By creating fewer but larger interfaces
 - C. By ensuring classes only implement methods they actually need
 - D. By eliminating the need for abstract classes
- Which example demonstrates adherence to ISP?
 - A. A *Printer* interface with methods *print*, *scan*, and *fax*
 - B. A *Vehicle* interface with methods for both *flying* and *driving*
 - C. Separate *Printer* and *Scanner* interfaces for *print* and *scan* methods

D. A *MediaPlayer* interface with methods for *play* and *record*

7 Dependency Inversion Principle

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- The Dependency Inversion Principle states:
 - A. High-level modules should depend on low-level modules
 - B. Low-level modules should depend on high-level modules
 - C. Both high-level and low-level modules should depend on abstractions
 - D. Abstractions should depend on details
- What is dependency injection in the context of the Dependency Inversion Principle?
 - A. Creating dependencies between two modules
 - B. Manually instantiating all objects within a class
 - C. Supplying dependencies to a class from outside, rather than creating them inside
 - D. Forcing a class to inherit from another

8 Overall Principles Questions

Bitte beantworten Sie die folgenden Multiple-Choice Fragen:

- What principle does the code below violate?
 - A. Open/Closed Principle
 - B. Liskov Substitution Principle
 - C. Single Responsibility Principle
 - D. Interface Segregation Principle

```

public class Invoice {
    private String customer;
    private double amount;

    public Invoice(String customer, double amount) {
        this.customer = customer;
        this.amount = amount;
    }

    public double calculateTax() {
        return amount * 0.2; // 20% tax
    }

    public void printInvoice() {
        System.out.println("Invoice for " + customer);
        System.out.println("Amount: " + amount);
        System.out.println("Tax: " + calculateTax());
    }
}

```

How could you refactor the above code?

- Which modification could fix the LSP violation in the code snippet, below?
 - A. Change *Penguin* to override fly with different behavior
 - B. Create a *NonFlyingBird* class that *Penguin* extends
 - C. Remove the *fly* method from *Bird*
 - D. Add more methods to *Penguin*

```

public class Bird {
    public void fly() {
        System.out.println("Bird is flying");
    }
}

public class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}

```


- Which principle is violated by the Worker interface in the code snippet, below?
 - A. Open/Closed Principle
 - B. Liskov Substitution Principle
 - C. Interface Segregation Principle
 - D. Dependency Inversion Principle

```
public interface Worker {  
    void work();  
    void eat();  
}  
  
public class OfficeWorker implements Worker {  
    @Override  
    public void work() {  
        System.out.println("OfficeWorker is working");  
    }  
  
    @Override  
    public void eat() {  
        System.out.println("OfficeWorker is eating");  
    }  
}  
  
public class Robot implements Worker {  
    @Override  
    public void work() {  
        System.out.println("Robot is working");  
    }  
  
    @Override  
    public void eat() {  
        throw new UnsupportedOperationException("Robot cannot eat");  
    }  
}
```

- How does the Notification class adhere to the Dependency Inversion Principle?
 - A. It directly creates an instance of *EmailService*
 - B. It depends on an abstraction (*MessageService*)
 - C. It uses inheritance instead of composition
 - D. It contains both high-level and low-level code

```
public interface MessageService {  
    void sendMessage(String message);  
}  
  
public class EmailService implements MessageService {  
    @Override  
    public void sendMessage(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}  
  
public class Notification {  
    private MessageService messageService;  
  
    public Notification(MessageService messageService) {  
        this.messageService = messageService;  
    }  
  
    public void notifyUser(String message) {  
        messageService.sendMessage(message);  
    }  
}
```